

Practical Work: Modular Arithmetic & Introduction to Cryptography

Mathematical Reasoning and Verification with Python

Pedagogical Sequence:

Modulo $\rightarrow$ Addition $\rightarrow$ Inverse Op. $\rightarrow$ Multiplication $\rightarrow$ Mult. Inverse $\rightarrow$ gcd $\rightarrow$ Valid Key $\rightarrow$ Enc/Dec
--

Instructions: Complete all calculations by hand on paper before using Python to verify your answers.

Exercise 1 — Understanding the Modulus

Consider the transformation:

$$C = (P + k) \bmod m \quad \text{with } P = 20, \quad k = 7, \quad m = 26$$

Questions

1. Before using Python, calculate C by hand.
2. What does $\bmod 26$ actually do to the result?
3. What possible set of values can C take?
4. Why does $m = 26$ make sense when working with the alphabet?
5. Verify your answer using Python.

Python Verification

```
1 P = 20
2 k = 7
3 m = 26
4
5 C = (P + k) % m
6 print(C)
```

Understanding Check

- Is $m = 26$ a secret key? Why or why not?

Exercise 2 — Modular Addition Encryption

Consider the scheme:

$$C = (P + k) \bmod m \quad \text{with } m = 26, \quad k = 7$$

Encrypt the following plaintext values: $P \in \{0, 1, 5, 10, 19, 20, 25\}$.

Questions

1. Calculate all ciphertexts without using Python.
2. Write a Python program to perform the encryption for all values.
3. Compare your manual calculations with the Python results.
4. What happens mathematically when $P + k \geq 26$?
5. Why does the ciphertext always stay in the range 0 to 25?
6. **Reflection:** If someone knows $m = 26$ and the encryption formula $C = (P + k) \bmod 26$, what piece of information do they still need to decrypt your message?

Exercise 3 — Discovering Decryption

Start with $C = (P + k) \bmod m$.

Questions

1. If encryption adds k , what mathematical operation should undo it? Derive the formula for P .
2. Using $m = 26$, $k = 7$, and ciphertext $C = 1$:
 - (a) Calculate P by hand.
 - (b) Verify your result using Python.
 - (c) Re-encrypt your recovered P . Does it produce $C = 1$?
 - (d) Why does subtraction undo addition modulo m ?
3. Complete the following Python functions:

```
1 def encrypt_add(P, k, m):  
2     return (P + k) % m  
3  
4 def decrypt_add(C, k, m):  
5     return (C - k) % m
```

Understanding Check

- Is the decryption key a completely different random number, or is it related to k ?

Exercise 4 — Modular Multiplication

Consider the encryption formula:

$$C = (P \times k) \bmod m \quad \text{with } P = 20, \quad k = 5, \quad m = 26$$

Questions

1. Calculate C by hand.
2. Verify using Python.
3. Evaluate the encryption function for all $P \in \{1, 2, 3, \dots, 25\}$.
4. Do different plaintext values ever produce the exact same ciphertext?
5. If two distinct plaintexts produce the same ciphertext, what problem does this create for decryption?

Exercise 5 — How Do We Divide Modulo m ?

Given $C = (P \times 5) \bmod 26$ and $C = 22$:

Questions

1. Why does $P = \frac{22}{5}$ not work directly in modular arithmetic over integers?
2. Find an integer x that satisfies:

$$5x \equiv 1 \pmod{26}$$

Search manually by testing $5(1), 5(2), 5(3), \dots \bmod 26$ until you find a remainder of 1.

3. State $5^{-1} \pmod{26}$.
4. Compute $P = (22 \times 5^{-1}) \bmod 26$ to recover the plaintext.

Python Verification

```
1 k = 5
2 m = 26
3
4 for x in range(m):
5     if (k * x) % m == 1:
6         print(f"The inverse of {k} mod {m} is: {x}")
```

Exercise 6 — Understanding the Multiplicative Inverse

For each pair (k, m) listed in Table 1, determine whether k has a multiplicative inverse modulo m .

k	m	$\gcd(k, m)$	Inverse $k^{-1} \pmod{m}$ exists? (Yes/No)
5	26		
7	26		
2	26		
13	26		
3	26		
5	25		

Table 1: Modular Inverse Compatibility

Questions

1. Fill in Table 1 by calculating $\gcd(k, m)$ for each pair.
2. For those that exist, find $k^{-1} \pmod{m}$ and verify that $k \times k^{-1} \equiv 1 \pmod{m}$.
3. Complete the fundamental rule:

k^{-1} exists modulo $m \iff \gcd(k, m) = \underline{\hspace{2cm}}$

Exercise 7 — Why Some Multiplication Keys Cannot Decrypt

Let $m = 26$ and $k = 2$. The encryption function is $C = (2P) \bmod 26$.

Questions

1. Calculate C for $P = 1, 2, 3, \dots, 13$.
2. Show that $E(1) = E(14)$.
3. If you receive ciphertext $C = 2$, can you uniquely determine whether the plaintext was 1 or 14?
4. Can this encryption function be safely decrypted?
5. Compute $\gcd(2, 26)$ and explain how this relates to the collision problem.

Exercise 8 — Build a Complete Multiplication Cipher

Consider the general modular multiplication cipher:

$$\text{Encryption: } C = (P \cdot k) \bmod m \quad \text{Decryption: } P = (C \cdot k^{-1}) \bmod m$$

Given parameters $m = 26$ and $k = 5$:

Tasks

1. Implement the encryption and decryption functions in Python:

```
1 def encrypt_mult(P, k, m):
2     # Your code here
3     pass
4
5 def decrypt_mult(C, k_inverse, m):
6     # Your code here
7     pass
```

2. Manually calculate $k^{-1} \pmod{26}$.
3. Encrypt $P = 20$, then decrypt the resulting ciphertext using k^{-1} . Verify that $P_{\text{recovered}} = 20$.

Exercise 9 — Key Generation

Let $m = 26$. We want to find all valid keys k such that $1 \leq k < 26$.

Questions

1. Write a Python script using `math.gcd` to find all k satisfying $\gcd(k, 26) = 1$.
2. How many valid keys exist for $m = 26$?
3. List the invalid key values. Why are they invalid?
4. For each valid k , compute its corresponding inverse $k^{-1} \pmod{26}$.

Exercise 10 — Sender and Receiver Scenario

Alice and Bob agree to communicate using a multiplication cipher with $m = 26$. Alice selects $k = 7$.

Questions

1. Is $k = 7$ a valid multiplication key modulo 26? Explain using gcd.
2. Calculate $k^{-1} \pmod{26}$.
3. Which key does Alice use to send messages, and which key does Bob use to read them?
4. Alice encrypts plaintext $P = 19$. Compute the ciphertext C .
5. Bob receives C . Perform the decryption calculation to show he recovers 19.
6. **Conceptual Question:** Does Bob need a completely unrelated secret key, or is his key directly derived from Alice's?

Exercise 11 — What Information Must Be Shared?

Consider the public system specifications:

- $m = 26$
- Encryption algorithm: $C = (P \times k) \bmod m$
- Secret key: $k = 5$

Questions

1. Which information can be made public without breaking the system?
2. Which information must remain secret?
3. If a receiver knows m and k , can they independently calculate k^{-1} ?
4. Does the sender need to explicitly transmit k^{-1} over the network to the receiver?

Exercise 12 — Comparing Addition and Multiplication Ciphers

Complete the comparative matrix in Table 2.

Property	Scheme A (Addition)	Scheme B (Multiplication)
Encryption Formula	$C = (P + 7) \bmod 26$	$C = (P \times 7) \bmod 26$
Decryption Formula		
Inverse Operation Required		
Is Every Key Reversible?		
Reversibility Condition		

Table 2: Comparison between Additive and Multiplicative Ciphers

Exercise 13 — Introduction to Modular Exponentiation

Consider the non-linear transformation:

$$C = P^k \bmod m \quad \text{with } P = 3, \quad k = 4, \quad m = 26$$

Questions

1. Compute $3^4 \bmod 26$ by hand.
2. Verify your answer using Python (`pow(3, 4, 26)`).
3. Compute C for $P \in \{1, 2, \dots, 25\}$ using $k = 4$.
4. Is reversing modular exponentiation as straightforward as subtracting k or multiplying by an inverse?
5. What mathematical problem must be solved to recover P from $P^k \equiv C \pmod{m}$?

Exercise 14 — Comprehensive Final Challenge

Scenario: Alice and Bob set up a multiplicative communication channel using $m = 29$, $k = 12$, and plaintext $P = 17$.

Part 1: Analytical Steps

- Key Validation:** Compute $\gcd(12, 29)$. Is $k = 12$ valid?
- Complementary Key:** Find $12^{-1} \pmod{29}$.
- Encryption:** Calculate $C = (17 \times 12) \bmod 29$.
- Decryption:** Calculate $P = (C \times 12^{-1}) \bmod 29$.
- Verification:** Show that $P_{\text{original}} = P_{\text{decrypted}}$.
- Conceptual Questions:**
 - Why does Bob specifically require 12^{-1} ?
 - Why isn't the modulus $m = 29$ kept secret?
 - What would happen if Alice had chosen $k = 2$ instead?
 - Does Bob need k^{-1} sent to him directly, or can he generate it himself?

Part 2: Complete System Implementation

Write a Python program that fulfills the following workflow:

1. Prompts the user for modulus m and key k .
2. Validates that $\gcd(k, m) = 1$. If invalid, rejects the key.
3. Computes $k^{-1} \pmod{m}$.
4. Prompts for a numerical plaintext P .
5. Performs encryption, prints C , performs decryption, prints $P_{\text{recovered}}$, and asserts equality.